PA consistency Turing machines

Comparison, exact checks, and a new reduction investigation

Research prepared for Vladimir Reshetnikov

Repository snapshot of 13 November 2026

Abstract

This investigation does not establish the requested machine with fewer than 196 working states. It compares every supplied incoming archive at a pinned repository revision, distinguishes duplicate presentations from different machines, constructs and checks new same-size zero-jump implementations, and rejects a new 195-state candidate by an exact counterexample to its claimed simulation. A separate constructor/parity integration gives a 202-state runtime with complete source-instruction contracts, but no certified blank startup. The archive includes executable checks and explicit limits on each result. It contains no table advertised as a verified sub-196 PA-consistency machine.

Contents

1 Problem, conventions, and reproducibility **2**

2 Comparison of the supplied designs **2**
2.1 Counting constructions without counting duplicates as savings . . . 2
2.2 Architectures and ancestry . . . 3
2.3 Why PA196 improves on PA209 . . . 3
2.4 All incoming archives . . . 4
2.5 Newer 196-state variants and failed 195-state reductions . . . 5

3 New zero-register jump specializations **6**
3.1 The inductive source invariant . . . 6
3.2 A complete same-size machine . . . 7
3.3 Synthesis at the actual entry context . . . 7
3.4 An exact rejection of a tempting 195-state table . . . 8

4 Other concrete approaches investigated **8**
4.1 Shorter scanner and source-tree geometry . . . 8
4.2 A fully integrated base-eight parity runtime . . . 9
4.3 What can be borrowed from ZFC and ConciseIPRA . . . 9

5 The exact consistency statement and proof levels **9**

A A base-four disequality recoding for parity acceptance **10**

B Reproduction, provenance, and limits **12**

1 Problem, conventions, and reproducibility

The requested target is an explicit deterministic, single-tape, two-symbol Turing machine with fewer than 196 working states such that

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \longleftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The tape is two-way infinite and initially zero everywhere. A transition reads one symbol, writes one symbol, moves one cell left or right, and enters the indicated control. The separate halting label is not a working state. For a table with n working states there must be precisely $2n$ rows. Changing the alphabet, supplying a prewritten tape, or counting a grammar nonterminal as if it were a physical control would change the problem.

All comparisons refer to the immutable ProveIt commit `1fe9b29810bfe3d0e507146d65f6b97692e7444a` [1, 2]. The PA subtree is `dc4d1ed9027c2b251ee751679d2690c6b1265b03`, and the incoming subtree is `19b0b7e26f84a3ebac927e8768c7a21067d03a06`. The archive manifest records the exact Git blob identities and SHA-256 hashes. This pins a finite collection; later changes to the repository are outside the audit.

The most useful separation is between four proof obligations:

1. The numerical calculus must be calibrated to ordinary PA, in both refutation directions when an internal biconditional is claimed.
2. The register program must implement a complete search of that calculus.
3. The literal transition table must simulate every relevant register instruction for arbitrary register values and arbitrary admissible tape gap.
4. The same table, started on a blank tape, must reach the specified invariant and source configuration.

Each obligation is necessary. A long nonhalting test of one blank run does not prove the third; the third does not prove the fourth. A correct local fold of an incorrectly patched parent does not establish the first two.

The package contains readable five-column transition tables, portable Python checkers, C++ search sources where relevant, machine-readable receipts, the archive census, this LaTeX source and the compiled PDF. The small verification entry points are described in `README.md`. No fresh complete Lean build of the upstream project is claimed. The report distinguishes inherited formal results, ordinary mathematical arguments, fresh finite certificates, and experimental constructions.

2 Comparison of the supplied designs

2.1 Counting constructions without counting duplicates as savings

The current incoming directory has 24 top-level ZIP archives. Recursively opening them, deduplicating identical archive bodies by SHA-256, gives 37 distinct ZIP bodies, 65 archive references, and nesting depth at most eight. Together with the PA directory, the inventory has 23,699 file occurrences. Its numeric-table parser recognizes 1,279 complete binary-TM table occurrences: 505 byte-distinct files and 382 root-free, action-preserving graph classes. These broad counts deliberately include ancestors, controls, failed experiments and repeated presentations. They are not a count of correct consistency machines.

The designated catalogue is more informative. Its 27 U208 presentations give 20 rooted transition graphs. The original nine give two graphs, with multiplicities five and four; eighteen later helper layouts give eighteen additional graphs. Seven explicit bijections were checked against

all 2,912 transition-row equations required for the original duplicate identifications. Renumbering states never contributes another state saving.

The selected 196-state catalogue contains eighteen rooted-nonisomorphic tables. Fourteen are the baseline or layouts reported as positive in their source packets; two are explicitly experimental MP layouts; two are rejected blank-halting controls. Five byte forms of supplied 195-state tables form four graph classes, all attached to rejected or diagnostic constructions. None is an accepted sub-196 answer in the supplied material. The complete identities and status labels are retained in the catalogue, so the counts can be inspected without relying on manuscript filenames.

2.2 Architectures and ancestry

Construction	States	Sites	DAG nodes	Max. path	Startup
Historical U208 A	208	3909	174	42	48140
Historical U208 B	208	3909	174	41	48140
Chang PA209	209	3948	173	33	58210
Chang PA196	196	3812	159	38	58154
Shorter-path alternative196	196	3812	159	36	58154
Combined196	196	3788	—	—	57922

Table 1: State counts exclude the halt label. Site, geometry and startup figures are those of the pinned constructions and their receipts; the graph identity census was rerun in this investigation. A dash means the summary does not assert a metric for that row.

The historical Joint U209 parent of U208 is different from Chang's PA209. Their similar sizes do not identify their source programs or native labels. The imported Chang PA196 revision [3] is `407cdc356f6b586635e71ac92be394255f4bfea4`; PA209 is `69d98241a721ead20b4f0b17df9062950b40fbfe`.

For PA196, a unary register value v is stored as the block $0\,1^{v+1}$. A source frame has a marker, a binary instruction-tree path, a sufficiently long zero gap, and the register file. The physical census is 36 arithmetic engine controls, 13 leaf routers, and 147 reader controls. The readers represent 159 distinct internal source nodes because some controls share different tree contexts. This sharing is why deleting two source nodes does not automatically delete two machine states.

The dense PA196 table has start 0, working controls $0, \ldots, 195$, and halt label 196. Its SHA-256 digest is

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The corresponding native labels are start 74, reader root 84, and halt 254. Dense root 75 is the same control. Native and dense labels must be translated through the included conjugacy, especially when comparing row patches from different packets.

2.3 Why PA196 improves on PA209

Let $p(a, b) = (a + b)(a + b + 1)/2 + b$. PA196 uses

$$I(a, b) = 4p(a, b), \quad M(a, b) = 4p(a, b) + 1, \quad U(a, B) = 4p(B, a) + 2, \quad E(a, b) = 4p(a, b) + 3.$$

Code 0 is truth and code 1 is the designated contradiction. The universal constructor reverses the pair components. The revised axiom emitter removes the old seventh equality head, places guarded equality vacuity in the gate, and changes head and antecedent order. Its 201 macro-token AX word has 68 binary grammar nodes. Six contextual reader folds reduce its physical body reader count to 62. Those six savings are already included in 196.

The complete source has 3,812 sites, compared with PA209's 3,948. The construction retains the command-search architecture and changes the emitter and its representation. The logical change is substantive: dropping an axiom without supplying its required consequences would invalidate the PA refutation compiler, even if every tape instruction continued to work.

The original U208 fold identifies its native controls 172 and 211. Its four exceptional local simulations have lengths 7, 1, 3, and 1, yielding

$$T_{208} = T_{209} + 6N(172, 0) + 2N(211, 0)$$

at aligned execution boundaries. The exact mixed-entry motif has no direct renaming into PA196: PA196 has no control whose two actions are $0Lq$ and $1Rq$ to the same working target. This rejects that direct motif, not every history-based reduction of PA196.

2.4 All incoming archives

The following table gives one row for each top-level incoming ZIP at the pinned commit. It summarizes contributions and limitations, while the machine-readable inventory retains every nested member.

Archive (ZIP basename)	Contribution	Limitation or outcome
`Below_U208_Research_2026-10-29`	Source-boundary windows, offset quotients and guarded source pruning	3,909→3,904 sites; no validated machine at or below 207
`ConPA_below208_research_2026-10-29`	Comparative audit, finite-model obstruction to cover-role identification, bounded fold searches	No reduction below 208; restricted minima are not global bounds
`PA_consistency_TM_research_2026-11-13`	Exact 159-node original source with zero-run bound 5; exact 157-node combined-negation/MP source; residual MP; broad local screens	No accepted sub-196 table; reader allocation and startup for the shorter scanner remain open
`U208_ConPA_2026-10-29`	Three-cell history fold 172/211	One original rooted U208 class; inherited theory calibration
`U208_ConPA_Conditional_Context_2026-10-29`	Conditional-entry proof of the fold; two presentations	The original two U208 classes, not extra independent savings
`U208_ConPA_Entry_Context_2026-10-29`	Complete entry-context proof and tables	Same primary/alternative rooted classes
`U208_ConPA_History_2026-10-29`	Finite-history presentation and exact local simulation	Same two original U208 classes
`conpa_tm_comparison_and_reduction_search`	MP without zero push and MP-result-in-H experimental 196 tables; register/layout catalogues	Two proposed Shift layouts actually halt after 43,563 blank steps; no sub-196 result
`conpa_tm_comparison_and_search`	A/D-register-swap 196 layout; rejected 195 quotient; logical countermodel	Rejected195 fails 93 of 4,959 source contracts
`conpa_tm_study_2026-11-13`	Combined196: H-only Shift and MP result in H; 3,788 sites and 4,928 contracts; direct-negation197 with 3,556 sites	Complete 196 intermediate, not a strict reduction; new logical endpoint is paper-level
`pa196_research_2026-11-13`	Exact 157-node representation of all 3,812 original CFG rows; HA-strength reference calculus	No correct sub-196 physical allocation; source-node savings lose contextual readers

Continued on next page

Archive (ZIP basename)	Contribution	Limitation or outcome
pa208_research_2026-10-29	Eighteen additional same-size U208 helper variants, source/theory/compiler work	Gives 20 total designated U208 classes, not a machine below 208
pa_consistency_comparison_2026_11_13	Modal-B plus explosion split of quantifier head; finite Hilbert proofs; 12 composition experiments; 189-token joint gate	Composition tables remain 196–200; shortened emitter is not a smaller machine
pa_consistency_reduction	Rebracketed196 and H-only196; integrated witness gate; ordinary internal-converse manuscript	Logical gate compiles to 203/206; rejected195 helper combination cycles
pa_consistency_research	Two exact 158-node source representations; repaired 196 physical layout; logical-head search	Nominal195 requires an eighth-zero scanner and becomes196
pa_consistency_research_2026-10-29	Inductively certified 3,909→3,904 source specialization	No validated physical machine below 208
pa_consistency_research_2026-11-13(2)	New rotated196; diagonal source improved 199→198; halt-at-successful-MP source; joint register abstraction	No strict improvement below 196; command source is not a new binary TM
pa_consistency_research_2026-11-13	Diagonal-vacuity calculus, 3,700-site, 199-state machine; short-negation197	4,815 symbolic contracts for 199; no sub-196 result
pa_tm_comparison_and_reductions	Rotated196 and rotated-macro62-196; distributed gate3,773 sites compiled to 198	Grammar-rule and macro-token counts are not state counts
pa_tm_extended_research_2026-11-13	Four closing-CD3/CD4/CD5/CD6 alternatives, stronger invariants, flexible symbolic checks	All 196; all four replace the same row and cannot be stacked
pa_tm_research_2026-11-13	Shorter-path196, independent symbolic checks, physical Lean subset, reverse-translation manuscript	Alternative lacks fresh full Lean endpoint; reverse arithmetic theorem remains manuscript-level
pa_unary_power_cover	Unary weak-covering theory and mutual PA interpretation/proof templates	No newly compiled literal machine
u208_comparison_research_2026-10-29	Original two-class census, weak/exact schema comparison, rejected207 frame witness	Does not include the later eighteen helper classes; no sub-208 result
u208_research_2026-10-29	Two U208 outputs and symbolic runtime evidence	Same original history-fold saving; no global minimum claim

2.5 Newer 196-state variants and failed 195-state reductions

The positive/baseline set includes the original PA196, the shorter-path layout, the repaired 158-node layout, two rotated layouts, rebracketed196, H-only196, A/D-swap196, four closing-decrement variants, a later twelve-rotation196, and Combined196. The four closing-decrement variants replace the same native row $(1365, 1)$; their savings cannot be added as if they were disjoint transformations.

Combined196 combines the H-only Shift test with MP result in H. It has 3,788 source sites, 4,928 uniform instruction contracts and a reported 57,922-step startup. Its table digest is

`bd74754d75e7b8ba97fa7bfc104374de33ae603fc183ae7f08f053ab8c9ea592`.

It is distinct from the earlier experimental MP-only 196 table.

The supplied rejected reductions fail for different reasons. One nominal 195 table loses its claimed reader refinement after a reassociation and fails 93 of 4,959 source contracts. Another

confuses a seven-zero source path with the marker; adding the missing scanner state returns the count to 196. A proposed helper combination enters an exact four-step loop from a source frame. A different dense $45 \to 160$ fold survives a long blank prefix but selects source site 615 instead of 605 at a certified later boundary. That witness refutes the proposed simulation, without proving that all possible analyses of the table have the same outcome.

The two rejected 196 Shift tables have a stronger failure: both actually halt from blank after 43,563 steps. Their unvisited difference at row $(115, 1)$ cannot repair that common finite trace. The repository contains a chunked Lean certificate for the trace. A finite blank halt and a failed simulation lemma should not be conflated.

3 New zero-register jump specializations

3.1 The inductive source invariant

Represent a register file by a nine-bit mask, whose bit r is set exactly when register r is positive. An increment sets its bit. At a zero decrement, the zero successor is taken without changing the mask. At a positive decrement, the positive successor is taken and its bit may remain set or become zero. A jump preserves the mask. These rules overapproximate the ordinary natural-number source execution.

Start the finite analysis at site 25 with mask 1, corresponding to $(2, 0, 0, 0, 0, 0, 0, 0, 0)$. Closing under the abstract transitions gives 274,880 site/mask pairs and 3,810 reachable sites. The two absent sites are 3774 and 3775. The checker explicitly verifies initial inclusion and closure after fixed-point discovery. Thus ordinary induction proves that every concrete source execution from the seed remains in the invariant. This is a finite inductive certificate, not a sampled run.

At fourteen unconditional forward-jump sites, at least one register is forced to zero. There are 43 individual choices of a register proved zero at such a site. Replacing the forward jump by the source decrement with the same zero successor preserves source execution at that boundary. Some isolated substitutions increase source DAG size; their counts are recorded as source counts rather than physical-state claims.

Only three whole physical reader rows can be changed uniformly using the common zero information in their represented occurrences:

Native row	Source sites	Common zero registers
$(27, 1)$	26	$3, 6, 7$
$(103, 1)$	$3, 9, 3665, 3671, 3683, 3689, 3766, 3772$	$3, 6$
$(1365, 1)$	3692	$3, 4, 5, 6, 7$

The distinction between an individual source occurrence and every context represented by one physical row is essential. A safe substitution at one source site does not authorize a global change to a row used elsewhere.

Proposition 3.1 (Zero-jump specialization)**.** *Suppose the checked mask invariant forces register r to zero at a source jump site i. Suppose the replacement decrement has the old jump destination as its zero successor. Replacing any collection of such sites does not change the sequence of source configurations reachable from the clean seed, apart from the names of the executed instructions.*

Proof. Use induction on source steps. At an unchanged site the step is identical. At a changed site the invariant gives $R_r = 0$, so the replacement takes its zero branch, leaves the whole register file unchanged, and reaches the old jump destination. The new and old configurations therefore agree, and membership in the original invariant is preserved. All ingredients are finite syntax and elementary arithmetic, so the same argument can be arithmetized in PA. □

3.2 A complete same-size machine

Changing only native row $(27, 1)$ of PA196 to

$$(1, R, 21)$$

implements the zero-decrement of logical register 6 at source site 26. In dense labels this is also $(27, 1) \mapsto (1, R, 21)$ for the pinned conjugacy. The source invariant forces that register to zero there. All 4,959 uniform instruction contracts pass when the changed site uses its certified zero precondition. The delivered table has SHA-256

`2c1a736399e0f32304ed18d6a439dc8d4fdd82f2ad103cdfc6851c0f4ae1e4af`.

Its rooted graph is distinct from all eighteen designated 196-state tables in the comparison catalogue. The unchanged arithmetic-engine rows continue to justify the register-box summaries. An exact literal blank simulation reaches the clean seed after 58,402 transitions.

Consequently this is a concrete new 196-state implementation of the same source search. Combining the mathematical source equivalence with the usual PA196 calibration transfers the intended consistency property at the ordinary proof level. This report does not identify a new exported Lean biconditional for this table.

3.3 Synthesis at the actual entry context

For each of the three rows, the search tries every write bit, both move directions, and every existing control or halt target. Each trial is checked against complete symbolic source contracts for the affected occurrences, with only the proved zero conditions added. It yields twelve valid actions for native $(27, 1)$, six for $(103, 1)$, and thirteen for $(1365, 1)$. The twelve source-26 actions also have exact clean-seed startups, ranging from 58,114 to 58,474 steps.

One-row validity does not imply joint validity: a replacement action may route through another row that a second replacement changes. Therefore the Cartesian product of the three action families is checked again at the whole-instruction level before attempting a fold. The final search receipt supplies the exact number of interacting combinations rejected and retained; no combination is called valid merely because its components were individually valid.

Composed search result. There are $12 \cdot 6 \cdot 13 = 936$ action combinations including the unchanged baseline. Among the 935 modified combinations, 264 are rejected by the full affected-contract checker after the action substitutions interact. This is a verification failure; no separate literal non-equivalence claim is made for every rejected combination. The remaining 671 pass those contracts. None yields a saving in the subsequent focused fold family: pairs touching a changed control with one identical projected row, one arbitrary helper-row repair, five-cell local simulations of at most sixteen steps, and contexts projected from the inductively closed radius-six blank-tape analysis. The recorded result is a finite exclusion within this template. It is not a lower bound of 196 states.

A separate, broader fold screen was run on the complete source-26 CD6 parent: all 38,220 ordered drop/keep pairs with two arbitrary retained rows, using five-cell windows and at most sixteen child steps per parent step. The global window abstraction gave no survivor. Intersecting it with contexts from all 4,959 source contracts and the exact startup reduced the projected set from 2,773 to 2,221 windows; the broader screen still gave no survivor after 573,273,344 second-row assignments. Computing the entire 274,880-pair register-mask invariant and applying its universally zero or positive coordinates, including unreachable branch elimination, produced 4,956 reachable symbolic instruction cases and exactly the same 2,221-window file. Residual

correlations between uncertain registers are still overapproximated. The equality of the resulting window files was checked by SHA-256, so another identical fold run was unnecessary.

3.4 An exact rejection of a tempting 195-state table

A preliminary patch sent native $(27, 1)$ to $(1, R, 62)$, on the assumption that control 62 was a context-independent decrement-7 entry. In that patched parent, native 27 and 46 can be merged while changing the retained zero row to $(0, L, 81)$. A local history-based check validates this fold relative to the patched parent. It does not validate the parent itself.

The complete source contract exposes the error. Control 62 is sensitive to the entry geometry: at source site 26 the proposed row implements the wrong operation. With path `100001`, gap 48, all nine registers zero and one spare of value zero, the original machine reaches the intended site 34 after 120 steps. Both the patched 196 parent and its 195 child reach site 27 after 558 steps, with register 6 equal to one and an extra spare. These are exact comparisons of finite tape configurations.

The included rejected table is therefore labelled as rejected, and its counterexample is independently replayable. In a focused repair attempt, 48 non-engine rows on the failing trace were allowed one arbitrary changed action. Of 37,584 actions tested, 153 repaired this single source contract, but none retained all other 4,958 contracts. This is a negative result for that repair family, not a nonexistence theorem about 195-state machines. A further targeted interaction changed the retained one-row and one row of each of five possible left-reading helpers. Of 15,680 alternatives, 113 repaired source 26, and none retained all 4,959 source contracts.

Nor can the wrong operation be silently accepted as a harmless change to word enumeration. In its altered scheduler, a positive continuation adds the candidate counter before decoding the next command. If a fresh candidate $n = p(0, b)$ starts with Shift and $b > 0$, the next decoded number is $n + b = p(2, b - 1)$. Its selector is therefore 2, an AX selector, rather than 0, a Shift selector. Thus no such fresh candidate starts with the old command prefix Shift–Shift. The inherited universal-word prefix lemma fails already at this short example. This does not rule out a different compiler for that scheduler, but no such compiler or admissible blank initializer has been supplied.

4 Other concrete approaches investigated

4.1 Shorter scanner and source-tree geometry

One supplied exact original-source representation has 159 internal nodes, maximum path 37, and maximum zero run five. It moves the seed path from `100000` to `10100000`. A smaller marker detector is attractive, but instruction selection temporarily changes address bits; a static zero-run bound alone does not prove that the dynamic scanner can be shortened. Some reader aliases are also lost when paths change.

In the focused scanner experiment included here, folding the extra scanner control into any retained control failed within the checked five-cell, 16-step and nine-cell, 32-step local simulation families. The broader family tested 195 retained controls, 869,456 first-row actions and 67,725,840 second-row actions. These numbers describe the implemented template; they neither bound the smallest TM nor exclude a redesigned frame convention. Source-DAG savings and a scanner-state saving cannot be added without validating their combined tape invariant and initializer.

4.2 A fully integrated base-eight parity runtime

A parity acceptance test is cheaper to express than an exact test for code one only if the coding makes every accepted odd formula refutable in PA. The ordinary base-four tags do not have that property: equality atoms have odd tag three and can be theorems. A parity test on the old coding is not a sound optimization.

The integrated prototype in this package instead uses base eight with tags $I = 0$, $M = 1$, $U = 2$, $E = 4$. Its constructor multiplier and all five equality-tag emissions are changed consistently. The Shift sum stays in the scratch register used by the parity guard, and the guard is matched to that revised source. This corrects the mismatch between a changed guard and an unchanged formula encoder.

The resulting AX word has 206 macro tokens. The integrated source has 4,062 sites. The raw table has 207 working states; five compatible reader folds give 202. Both tables pass all 5,205 uniform symbolic source contracts. The folded table digest is

`d9a0256666bbd6e5cad616e5b2df7f56b2773b86df75842fd2ba17ccb45a3f76`.

These are instruction-runtime checks, not a proof of the revised command arithmetic or logical proof compiler. The included structural audit confirms that MP reconstructs its implication with the altered multiplier; a complete command-calibration theorem remains a separate obligation. This is a verified runtime implementation, not a completed blank-start consistency machine: no certified sound scheduler seed was obtained from its blank run. A finite failure to find a seed is not a theorem that no such seed ever occurs. In any event, 202 is above the requested threshold.

4.3 What can be borrowed from ZFC and ConciseIPRA

The ZFC K215 material [4] gives useful implementation methods: command-local field readers, partial selectors, folds using the preceding tape bit, and explicit local stuttering simulations. Its independent runtime and startup obligations are particularly relevant to the source-26 error above. However, its 215-state table implements a different numerical program and calculus. An optimization of its decoder is not automatically available in PA196 merely because native control numbers coincide.

ConciseIPRA [5] separates a concise scalar syntax and a normalized proof checker from their arithmetical interpretation. That is a useful design principle for a replacement proof-search architecture. It does not justify substituting consistency of iPRA for consistency of PA. A smaller checker would need explicit PA-refutation translation in both directions and a complete binary tape implementation. No unproved equality of theory strength is used in this report.

5 The exact consistency statement and proof levels

The imported PA196 endpoint explicitly proves

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

It also establishes external nonhalting and independence in the Lean metatheory. An external soundness theorem does not, solely by its existence, supply the reverse implication inside PA. Two-sided independence of a sentence likewise does not determine its equivalence class over PA.

The incoming manuscripts contain more than an unsupported assertion of the reverse direction. They describe a syntactic interpretation of membership as binary-bit membership, primitive equality as equality, and the transposed universal code as the corresponding quantifier. Their

finite comprehension proof uses one PA induction instance formed from the input formula. They handle the code-zero truth alias and the special case where the two labels in the guarded gate coincide. These are mathematical arguments that can be evaluated independently of whether a new Lean endpoint has been compiled.

The essential proof-producing statement has the form

$$\mathrm{PA} \vdash \mathrm{Prf}_C(p, 1) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}\big(F(p), \ulcorner \bot \urcorner\big),$$

where F translates a finite numerical-calculus derivation into a PA refutation. A proof certificate is carried with each live formula and each stack entry. AX introduces certified axioms, MP combines certificates, generalization constructs its proof, and stack underflow returns certified truth. At the original Shift halt, $t + h = 1$, so one certified live formula has code one.

To pass from an actual tape halt to this certificate, use the exact finite startup and total positive-length instruction simulations. Determinism aligns the literal trace with the source run. A halting trace of length N cannot contain more than N completed source instructions. Internal engine paths must have their specified total exits and no unexpected halt. This yields a bounded trace extraction followed by the proof translation. The argument uses syntactic proof construction, not a uniform PA truth predicate or an assumed PA reflection principle.

The current repository's `reverse-cert-stack/` files add actual standard-natural-number certificate-stack and native-checker bindings. They address real parts of the proof, including zero aliases and ordering of the imported command heads. They do not yet export the all-model literal-trace bridge and the exact model-valued checker binding needed for a closed formal internal biconditional. The `ipra/` bridge is also conditional on the machine-specific reductions.

For a literal new table, a mathematically complete claim therefore requires both the syntactic consistency calibration and the full source/tape/startup transport. A missing Lean endpoint is a formalization limitation; a failed tape contract is a mathematical failure of the proposed construction. The package does not blur these two cases.

A A base-four disequality recoding for parity acceptance

This section gives a new high-level consistency calibration. It is not a transition table with fewer than 196 states. The point of the construction is that parity acceptance can be made sound without changing the existing Cantor-pairing multiplier or the modus-ponens decoder.

Write $p(a, b) = (a + b)(a + b + 1)/2 + b$ and retain

$$I(a, b) = 4p(a, b), \qquad M(a, b) = 4p(a, b) + 1, \qquad U(a, A) = 4p(A, a) + 2.$$

Interpret the remaining constructor

$$D(a, b) = 4p(a, b) + 3$$

as disequality. Let $N(A) = I(A, 1)$ and use the defined equality expression $E^*(a, b) = N(D(a, b))$ throughout the PA196 axiom families and guarded gate. Code zero remains truth, and code one remains the contradiction $M(0, 0)$. Keep modus ponens, closed generalization, and the original first quantifier head. In particular the explosion principle remains derivable; it is not being inferred merely from an intended interpretation of code one.

The six heads and the gate are precisely those in the pinned PA196 paper with each displayed primitive equality replaced by E^*. One may use the classically equivalent shorter witness head

$$N\big(U(d, D(h, d))\big)$$

in place of $\exists x_d\, E^*(h, d)$. Their equivalence follows by propositional double-negation elimination/introduction, generalization, and the quantifier-distribution head. It is consequently a finite proof-template change, not a new semantic assumption.

Soundness of odd-code acceptance. Translate $M(a, b)$ to arithmetic binary-bit membership and $D(a, b)$ to $x_a \neq x_b$, with implication and universal quantification translated as usual. In this translation E^* is equivalent to ordinary equality. Thus the same PA proof templates for the six heads, guarded vacuities, and pointed finite separation apply. Freshness is unchanged: every label in the decoded formula of positive code c is below c, so the gate's variable x_c is fresh in its property.

Every odd code is either a membership atom or a disequality atom. After replacing all its free variables by zero, its translation is respectively $\mathrm{Bit}(0, 0)$ or $0 \neq 0$. PA refutes both. A derivation of an odd code therefore translates to a PA refutation: first construct a PA proof of the translated formula, then apply uniform zero substitution and append the fixed refutation. This is a syntactic proof transformation; no uniform truth predicate is used.

Consequently either a cached-head parity test or a test that the sum of the two certified live formulas is odd is sound relative to PA consistency. In the latter case an odd sum has an odd summand, whose retained derivation supplies the refutation. The original base-four encoding cannot justify this step, since its equality atoms have odd residue three and need not be false at zero.

A precise transport of the old calculus. Define τ by course-of-values recursion. Put $\tau(0) = 0$. For $n > 0$, write $n = 4p(a, b) + r$ and set

$$\begin{aligned}
\tau(4p(a, b)) &= I(\tau(a), \tau(b)),\\
\tau(4p(a, b) + 1) &= M(\tau(a), \tau(b)),\\
\tau(4p(a, b) + 2) &= 4p(\tau(a), \tau(b)) + 2,\\
\tau(4p(a, b) + 3) &= N(D(\tau(a), \tau(b))).
\end{aligned}$$

The third line preserves the already-transposed universal constructor: $\tau(U(a, A)) = U(\tau(a), \tau(A))$. Also $\tau(1) = 1$.

The map is primitive recursive and injective. For injectivity, observe first that no value of τ has residue three modulo four. Images of membership and universal codes have distinct residues one and two. Images of positive implication codes and of old equality codes have residue zero, but they cannot collide: the antecedent component of an old equality image is the disequality code $D(\tau(a), \tau(b))$, with residue three, whereas the antecedent of an implication image is itself in the range of τ. Two images in the same case reduce to equality of smaller component images by injectivity of Cantor pairing. Zero is separate because a positive implication image can be zero only if both original components were zero, contrary to positivity, and an equality image has consequent one. This proves the claim by induction.

Both variable labels and formula codes are transported by this same map. This detail is essential: the gate uses its property parameter c also as the bound-variable label. Injectivity preserves both freshness inequalities. The translated equality heads and gate are exactly the new families, before the optional witness simplification. Modus ponens and generalization transport directly. For the restricted propositional schema, use its already-derived unrestricted version; its finite construction uses the usual component bounds for I, not monotonicity of τ in the natural-number order. Thus every old proof has a primitive-recursively constructed new proof, and the transformation's correctness is arithmetizable in PA.

Composing with the PA196 PA-refutation compiler gives a derivation of new code one from every PA refutation. The source command compiler must be updated for the new emitted formulas, but the AX push count is unchanged. All AX-generated heads and the gate have even top tags, as does the generalization entry. The usual command proof permits early halting under the stronger acceptance test, and a completed derivation of code one is accepted. Subject to that

explicit source compiler and its tape simulation, the intended result is

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(R_{\mathrm{diseq,parity}}) \longleftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The argument supplies the calculus and acceptance calibration; it does not assert that a new binary table and full PA-internal compiler proof have been implemented.

The retained finite emitter experiment. The standard equality-vacuity PA196 AX word has 201 macro tokens and 68 binary macro-tree nodes. Replacing equality by negative disequality and performing the witness double-negation simplification gives 210 tokens. The six head-emitter token counts are $17, 19, 19, 19, 13, 19$; the new gate uses 83 tokens. A beam-limited parse over subtrees of the inherited source has 75 binary macro-tree nodes. Its balanced assembly exceeds an inherited zero-run constraint and is not a verified physical layout. A separate 30,000-trial randomized RePair screen over equality and membership orientation, three antecedent exchanges, gate order, and head order found a best grammar with 75 internal nodes. These are heuristic upper bounds, not lower bounds, and no source-node count is presented as a TM working-state count. The retained scripts and grammar witnesses reproduce the exact finite arithmetic-expression experiments.

B Reproduction, provenance, and limits

The archive preserves attribution to James Chang and to the ProveIt and incoming research sources. The original tables and proof files are inputs, not new results of this investigation. New scripts and receipts are identified separately. SHA-256 checks protect the exact transition tables, source program and archive members; the final package manifest records the files delivered here.

The default verification commands check the curated graph comparison, the zero-register source invariant, the chosen complete 196-state jump variant, its exact startup, the finite rejection witness, and the base-eight runtime integration. Larger finite search replays are separate because their running times and compiler availability differ. Search receipts record their actual scope, parameters and result counts.

The inventory does not execute arbitrary submitted archive code. It parses files and validates table structure and graph identities. The new source and runtime checks use the explicitly packaged code and pinned inputs. No negative result in this report is asserted to exhaust all binary Turing machines, all state encodings, all source transformations, or all possible mathematical consistency encodings.

References

[1] V. Reshetnikov, *ProveIt*, pinned revision `1fe9b29810bfe3d0e507146d65f6b97692e7444a`. `https://github.com/VladimirReshetnikov/ProveIt/tree/1fe9b29810bfe3d0e507146d65f6b97692e7444a/Computability/BusyBeaver/Research/PA`.

[2] *Incoming research archives*, same revision. `https://github.com/VladimirReshetnikov/ProveIt/tree/1fe9b29810bfe3d0e507146d65f6b97692e7444a/docs/incoming`.

[3] J. Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, imported PA196 paper and Lean development, upstream revision `407cdc356f6b586635e71ac92be394255f4bfea4`.

[4] *ZFC research, candidate K215*, ProveIt, same pinned revision. `https://github.com/VladimirReshetnikov/ProveIt/tree/1fe9b29810bfe3d0e507146d65f6b97692e7444a/Computability/BusyBeaver/Research/ZFC`.

[5] *ConciseIPRA*, ProveIt, same pinned revision. `https://github.com/VladimirReshetnikov/ProveIt/tree/1fe9b29810bfe3d0e507146d65f6b97692e7444a/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.